Cuprins

1 Pseudocod si variabile **2**

2 Tehnici si fenomene asociate: Detalii matematice si demonstratii **4**
2.1 Conversie la grayscale 4
2.2 Blur: filtrare spatiala si bilaterala 4
2.3 Thresholding adaptiv (Otsu) 4
2.4 Detectarea de margini (calcul de gradienti) 4
2.5 Regresie Liniara / metoda celor mai mici patrate 5
2.6 Calcul Roll si Pitch 5

3 Implementare Python: structura si etape **6**
3.1 crop_and_scale.py 6
3.2 draw_display.py 7
3.3 find_horizon.py 8

4 Exemple **13**
4.1 Orizont detectabil 13
4.2 Orizont nedetectabil 14

1 Pseudocod si variabile

Algoritmul poate fi descris prin urmatorul pseudocod:

```
Input:  frame (imagine color),
        exclusion_thresh (prag de filtrare),
        FOV (camp vizual),
        acceptable_variance (dispersie maxima acceptata)

Output: roll, pitch, variance, is_good_horizon, mask


# 1. Reducerea complexitatii si izolarea regiunii cerului
    gray           <- conversie grayscale a frame
    hsv            <- conversie HSV a frame
    blue_mask      <- filtrare interval nuante de albastru (lower, upper)
    blue_filtered  <- gray + blue_mask
    blur           <- aplicare filtru bilateral pe blue_filtered
                      # pastreaza marginile, reduce zgomot

# 2. Evidentierea marginilor de separatie cer/teren
    mask           <- aplicare threshold Otsu pe blur
                      # masca binara, prag adaptiv T calculat automat
    edges          <- aplicare Canny pe gray
                      # detecteaza discontinuitatile de intensitate

# 3. Eliminarea zgomotului si a obiectelor irelevante
    contours       <- gasire contururilor in mask
    daca contours este gol:
        return valori prezise anterior sau None

    largest_contour <- contur cu aria maxima
    extrage puncte x, y din largest_contour
    separa puncte margine (x_edge, y_edge)
           si puncte interne (x_inner, y_inner)

# 4. Aproximarea liniei orizontului
    daca predicted_roll exista:
        filtreaza puncte interne dupa exclusion_thresh
        fata de linia prezisa

    (m, b)        <- regresie polinomiala y = m x + b
                     # metoda celor mai mici patrate
    sky_is_up     <- determinare pe baza punctelor de margine

# 5. Calculul unghiurilor si estimarea stabilitatii
    roll          <- arctan(m)
    d             <- distanta perpendiculara de la centrul imaginii
                     la fit (produs vectorial 2D)
```

```
pitch         <- d / H * FOV
variance      <- media distantelor punctelor fata de fit
is_good_horizon <- (variance < acceptable_variance)

actualizeaza predicted_roll, predicted_pitch pentru urmatorul cadru

return roll, pitch, variance, is_good_horizon, mask
```

Explicatii suplimentare:

- *exclusion_thresh* reprezinta toleranta in grade pentru indepartarea punctelor care se abat de la predictia anterioara a liniei orizontului.
- *acceptable_variance* este pragul care determina daca fit-ul este stabil si sigur.
- *mask* contine regiunile binare de cer detectate, util pentru vizualizare si diagnostic.
- Filtrarea bilaterala permite reducerea zgomotului pastrand discontinuitatile, esential pentru mentinerea marginii orizontului.
- Regresia liniara minimizeaza eroarea patratica intre punctele detectate si linia aproximativa $y = mx + b$.
- Roll reprezinta inclinatia camerei in jurul axei longitudinale (rotatia spre stanga/-dreapta), iar Pitch reprezinta deviatia verticala a liniei orizontului fata de centrul imaginii (camera priveste in sus sau in jos). Aceste unghiuri sunt determinate prin raportarea pozitiei liniei $y = mx + b$ fata de centrul imaginii si folosind distanta perpendiculara de la centru la linie.

2 Tehnici si fenomene asociate: Detalii matematice si demonstratii

2.1 Conversie la grayscale

Grayscale-ul se obtine ca o combinatie liniara a canalelor RGB. Coeficientii 0.299, 0.587 si 0.114 provin din perceptia umana a luminozitatii. Demonstrativ, luminozitatea perceputa L poate fi aproximata ca medie ponderata a componentelor, unde verdele contribuie cel mai mult:

$$L = 0.299R + 0.587G + 0.114B.$$

Aceasta derivare porneste de la standardul BT.601, care normalizeaza intensitatea astfel incat media valorilor ponderate corespunde perceptiei vizuale.

2.2 Blur: filtrare spatiala si bilaterala

Se poate demonstra: daca $I(p) \approx I(q)$ pentru vecini apropiati, factorul de intensitate este aproape 1, astfel incat media ponderata nu modifica valoarea centrala. sa luam un pixel central p cu valoarea $I(p)$ si vecini $q \in \mathcal{N}(p)$:

$$I_{\text{out}}(p) = \frac{1}{W_p} \sum_{q \in \mathcal{N}(p)} I(q) \cdot e^{-\frac{||p-q||^2}{2\sigma_s^2}} \cdot e^{-\frac{|I(p)-I(q)|^2}{2\sigma_r^2}}.$$

Daca $I(p) \approx I(q)$, exponentul pe intensitate $e^{-\frac{|I(p)-I(q)|^2}{2\sigma_r^2}} \approx 1$. Atunci contributia pixelilor vecini este proportionala doar cu distanta spatiala, deci valoarea centrala $I(p)$ se pastreaza aproape neschimbata. Daca exista o discontinuitate (ex. linia orizontului), $|I(p) - I(q)|$ este mare, exponentul scade si vecinii care ar altera marginea au contributie neglijabila. Aceasta arata inductiv ca filtrul reduce zgomotul fara a estompa marginile.

2.3 Thresholding adaptiv (Otsu)

Otsu cauta pragul T care minimizeaza dispersia intra-clasa:

$$\sigma_w^2(T) = w_0(T)\sigma_0^2(T) + w_1(T)\sigma_1^2(T),$$

unde $w_0 = \frac{N_0}{N}$, $w_1 = \frac{N_1}{N}$, iar $\sigma_0^2 = \frac{1}{N_0}\sum_{i \in C_0}(I_i - \mu_0)^2$, $\sigma_1^2 = \frac{1}{N_1}\sum_{i \in C_1}(I_i - \mu_1)^2$. Demonstrativ, derivata lui $\sigma_w^2(T)$ fata de T setata la zero gaseste punctul unde dispersia combinata a claselor este minima, garantand separarea optima a clusterelor de pixeli pentru cer si teren.

2.4 Detectarea de margini (calcul de gradienti)

Gradientul este derivata (unde I - Intensitate):

$$G(x, y) = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}.$$

Prin diferenta finita, $\frac{\partial I}{\partial x} \approx I(x+1, y) - I(x, y)$. Demonstrativ, pentru o schimbare brusca a intensitatii, derivata devine mare, marcand marginea. Maximul local al G corespunde punctului de discontinuitate.

2.5 Regresie Liniara / metoda celor mai mici patrate

Fie n puncte (x_i, y_i). Fit-ul $y = mx + b$ minimizeaza

$$E(m, b) = \sum_{i=1}^{n} (y_i - mx_i - b)^2.$$

Derivand:

$$\frac{\partial E}{\partial m} = -2 \sum (y_i - mx_i - b)x_i = 0, \quad \frac{\partial E}{\partial b} = -2 \sum (y_i - mx_i - b) = 0,$$

rezulta formulele:

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}, \quad b = \bar{y} - m\bar{x}.$$

Demonstrativ, derivarea arata ca reziduurile verticale trebuie sa fie mediate la zero, iar panta m maximizeaza corelatia liniara intre x si y.

2.6 Calcul Roll si Pitch

Pentru Roll, avem fit-ul $y = mx + b$. Panta m reprezinta tangenta unghiului de inclinatie fata de orizontala. Aplicand definitia tangentei in geometria plana:

$$\tan(\text{Roll}) = m \quad \Rightarrow \quad \text{Roll} = \arctan(m).$$

Pentru Pitch, consideram doua puncte de pe fit: (x_1, y_1) si (x_2, y_2) si centrul imaginii (x_c, y_c). Distanta perpendiculara de la punct la linie se obtine folosind produsul vectorial in 2D. Fie $\vec{v} = (x_2 - x_1, y_2 - y_1)$ vectorul liniei si $\vec{w} = (x_c - x_1, y_c - y_1)$ vectorul de la un punct de pe linie la centrul imaginii. Distanta perpendiculara este:

$$d = \frac{|\vec{v} \times \vec{w}|}{||\vec{v}||} = \frac{|(x_2 - x_1)(y_1 - y_c) - (x_1 - x_c)(y_2 - y_1)|}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}.$$

Aceasta rezulta din definitia produsului vectorial in planul 2D, care masoara aria paralelogramului format de cei doi vectori; raportand aceasta la lungimea vectorului liniei obtinem distanta perpendiculara de la punct la linie.

In final, Pitch-ul se exprima in grade fata de centrul imaginii prin scalare cu campul vizual al camerei:

$$\text{Pitch} = \frac{d}{H} \cdot FOV,$$

unde H este inaltimea imaginii in pixeli, iar FOV este unghiul de camp al camerei. Aceasta relatia este exact derivata din similaritatea triunghiurilor: fit-ul este considerat ca baza, iar deviatia verticala a centrului imaginii este proportionala cu unghiul sub care linia este vizibila.

3 Implementare Python: structura si etape

Algoritmul este organizat in trei fisiere principale, fiecare cu rol clar definit si interdependente logice: `crop_and_scale.py`, `draw_display.py` si `find_horizon.py`.

3.1 crop_and_scale.py

Gestioneaza pregatirea imaginilor pentru inferenta. Este responsabil cu:

- Calcularea parametrilor de decupare pentru a mentine aspect ratio-ul dorit.

```
new_aspect_ratio = new_resolution[0] / new_resolution[1]
original_aspect_ratio = original_resolution[0]/original_resolution[1]
if new_aspect_ratio > original_aspect_ratio:
    new_aspect_ratio = original_aspect_ratio
```

- Redimensionarea imaginii la rezolutia de inferenta.

```
width, height = original_resolution
new_width = height * new_aspect_ratio
margin = (width - new_width) // 2
cropping_start = margin
cropping_end = width - margin
```

Aplicarea scalarii uniforme:

```
scale_factor = new_resolution[1] / height
frame_cropped = frame[:, cropping_start:cropping_end]
frame_scaled = cv2.resize(frame_cropped, (0,0), fx=scale_factor,
        fy=scale_factor)
```

Etape:

1. Se determina aspect ratio-ul al imaginii originale si al rezolutiei tinta.
2. Se decide daca trebuie ajustat aspect ratio-ul prin decupare centrata.
3. Se calculeaza coordonatele de start si end pentru decupare pe axa orizontala.
4. Se aplica scalarea imaginii pentru a se potrivi rezolutiei tinta, folosind un factor de scalare uniform.

Important: Acest pas reduce computatiile si garanteaza consistenta dimensiunilor intre cadre, esential pentru algoritmul de detectare si filtrare ulterioara.

3.2 draw_display.py

Se ocupa de reprezentarea vizuala a orizontului detectat si a elementelor auxiliare pe imagine. Este independent de detectia efectiva, dar esential pentru diagnostic si validare vizuala.

Rolul principal: trasarea liniei orizontului pe cadru conform valorilor de *roll* si *pitch* calculate.

Etape:

1. Transformarea *roll* din grade in radiani.

```
roll = radians(roll)
```

2. Determinarea orientarii cerului (sus/jos) pe baza *roll*-ului.

```
sky_is_up = (roll >= FULL_ROTATION_RADIANS * 0.75
             or (roll > 0 and roll <= FULL_ROTATION_RADIANS * 0.25))
```

3. Calcularea pozitiei liniei perpendiculare fata de centrul imaginii folosind *pitch* si campul vizual.

```
distance = pitch / fov * frame.shape[0]
angle_perpendicular = roll + pi / 2
x_perpendicular = distance * cos(angle_perpendicular) + frame.shape[1] / 2
y_perpendicular = distance * sin(angle_perpendicular) + frame.shape[0] / 2
```

4. Determinarea pantei si intersectiei liniei (m, b) pentru trasarea pe margini.

```
run = cos(roll)
rise = sin(roll)
if run != 0:
    m = rise / run
    b = y_perpendicular - m * x_perpendicular
else:
    m = None
```

5. Trasarea liniei principale a orizontului si, optional, a liniei de sol perpendiculare pentru referinta.

```
p1, p2 = _find_points(m, b, frame.shape)
cv2.line(frame, p1, p2, color, 2)

if draw_groundline and m != 0:
    m_perp = -1 / m
    b_perp = y_perpendicular - m_perp * x_perpendicular
    points = _find_points(m_perp, b_perp, frame.shape)
    p1x = int(np.round(x_perpendicular))
    p1y = int(np.round(y_perpendicular))
    p1 = (p1x, p1y)
    cv2.line(frame, p1, points[1], (0,191,255), 1)
```

Observatie: Functia `_find_points` calculeaza intersectiile liniei cu marginile imaginii, asigurand trasarea completa chiar daca panta este mare sau zero. Acest modul faciliteaza intelegerea vizuala a detectarilor si ajustarea parametrilor.

3.3 find_horizon.py

Implementeaza detectarea efectiva a liniei orizontului. Functioneaza pe imaginea pregatita (decupata si scalata) si realizeaza intreaga secventa de preprocesare, selectie si regresie liniara.

Etape:

1. **Preprocesare:** conversie in grayscale, filtrare a nuantelor de albastru (cer) si filtrare bilaterala pentru reducerea zgomotului.

```
bgr2gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
lower = np.array([109, 0, 116])
upper = np.array([153, 255, 255])
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
hsv_mask = cv2.inRange(hsv, lower, upper)
blue_filtered_greyscale = cv2.add(bgr2gray, hsv_mask)
blur = cv2.bilateralFilter(blue_filtered_greyscale, 9, 50, 50)
```

2. **Thresholding si Canny edge detection:** obtinerea marginilor si a mascii binare care evidentiaza zona cerului.

```
_, mask = cv2.threshold(blur, 250, 255, cv2.THRESH_OTSU)
edges = cv2.Canny(image=bgr2gray, threshold1=200, threshold2=250)
edges = skimage.measure.block_reduce(edges, (KERNEL_SIZE, KERNEL_SIZE), np.max)
```

3. **Detectarea contururilor:** extragerea celui mai mare contur pentru a identifica punctele candidate pentru linia orizontului.

```
contours, _ = cv2.findContours(mask, cv2.RETR_TREE, cv2.CHAIN_APPROX_NONE)
if len(contours) == 0:
    self._predict_next_horizon()
largest_contour = sorted(contours, key=cv2.contourArea, reverse=True)[0]
x_original = np.array([i[0][0] for i in largest_contour])
y_original = np.array([i[0][1] for i in largest_contour])
```

4. **Subsampling si separarea punctelor:** reducerea numarului de puncte pentru viteza si separarea punctelor de margine de cele interne, pentru determinarea orientarii cerului.

```
x_abbr, y_abbr = [], []
x_edge_points, y_edge_points = [], []
for n, x_point in enumerate(x_original):
    y_point = y_original[n]
    if x_point==0 or x_point==frame.shape[1]-1 or y_point==0 or
    y_point==frame.shape[0]-1:
        x_edge_points.append(x_point)
        y_edge_points.append(y_point)
    else:
        x_abbr.append(x_point)
        y_abbr.append(y_point)
# Subsampling pentru viteza
max_points = 20
step_size = len(x_edge_points) // max_points
if step_size > 1:
    x_edge_points = x_edge_points[::step_size]
    y_edge_points = y_edge_points[::step_size]
max_points = 100
step_size = len(x_abbr) // max_points
if step_size > 1:
    x_abbr = x_abbr[::step_size]
    y_abbr = y_abbr[::step_size]
```

5. **Filtrare dupa predictia anterioara:** daca exista valori precedente de *roll* si *pitch*, se elimina punctele care se abat prea mult de la linia prezisa, pentru stabilitate.

```
x_filtered = []
y_filtered = []
if self.predicted_roll is None:
    x_filtered = x_abbr
    y_filtered = y_abbr
else:
```

```
    p1 = np.array([0, predicted_b])
    p2 = np.array([frame.shape[1], predicted_m * frame.shape[1] + predicted_b])
    p2_minus_p1 = p2 - p1
    for idx, x_point in enumerate(x_abbr):
        y_point = y_abbr[idx]
        p3 = np.array([x_point, y_point])
        distance = norm(np.cross(p2_minus_p1, p1 - p3) / norm(p2_minus_p1))
        if distance < self.exclusion_thresh:
            x_filtered.append(x_point)
            y_filtered.append(y_point)
x_filtered = np.array(x_filtered)
y_filtered = np.array(y_filtered)
```

6. **Regresie liniara:** aplicarea metodei celor mai mici patrate pentru obtinerea parametrilor liniei $y = mx + b$.

```
m, b = np.polyfit(x_filtered, y_filtered, 1)
```

7. **Calcul Roll si Pitch:** determinarea unghiului de rotatie (*roll*) si a deviatiei verticale (*pitch*) fata de centrul imaginii, folosind geometrie plana si produs vectorial.

```
roll = degrees(atan2(m, 1))
p1 = np.array([0, b])
p2 = np.array([frame.shape[1], m*frame.shape[1]+b])
p3 = np.array([frame.shape[1]//2, frame.shape[0]//2])
distance_to_horizon = norm(np.cross(p2 - p1, p1 - p3)) / norm(p2 - p1)
pitch = distance_to_horizon / frame.shape[0] * self.fov
if not plane_pointing_up:
    pitch *= -1
```

8. **Calcului dispersiei:** masurarea distantei punctelor fata de fit pentru a estima increderea in detectare.

```
p2_minus_p1 = p2 - p1
distance_list = [norm(np.cross(p2_minus_p1, p1 -

np.array([x_filtered[n], y_filtered[n]]))) / norm(p2_minus_p1)
                 for n in range(len(x_filtered))]
variance = np.average(distance_list) / frame.shape[0] * 100
```

9. **Predictia urmatorului cadru:** extrapolarea liniara a valorilor *roll* si *pitch* pentru filtrarea zgomotului in urmatorul cadru.

```python
self._predict_next_horizon(roll, pitch, is_good_horizon)

def _predict_next_horizon(self, current_roll = None, current_pitch = None, is_goo
        """
            Based on the positions of recent horizons, predict the approximate
            position of the next horizon.
            Used to filter out noise in the next iteration.
        """
        if not is_good_horizon:
            current_horizon = None
        else:
            current_horizon = (current_roll, current_pitch)

        self.recent_horizons.append(current_horizon)
        del self.recent_horizons[0]

        if None in self.recent_horizons:
            self.predicted_roll = None
            self.predicted_pitch = None
        else:
            roll1 = self.recent_horizons[0][0]
            roll2 = self.recent_horizons[1][0]
            roll_delta = roll2 - roll1
            self.predicted_roll = roll2 + roll_delta

            pitch1 = self.recent_horizons[0][1]
            pitch2 = self.recent_horizons[1][1]
            pitch_delta = pitch2 - pitch1
            self.predicted_pitch = pitch2 + pitch_delta
```

10. **Ajustare Roll:** normalizarea unghiului in intervalul $[0, 360]^\circ$ in functie de orientarea cerului.

```python
roll = self._adjust_roll(roll, sky_is_up)

def _adjust_roll(self, roll: float, sky_is_up: bool) -> float:
        """
        Adjusts the roll to be within the range of 0-360 degrees.
        Removes negative values and values greater than 360 degrees.
        """
        roll = abs(roll % FULL_ROTATION)
        in_sky_is_up_sector = (roll >= FULL_ROTATION * .75 or
            (roll > 0 and roll <= FULL_ROTATION * .25))

        if sky_is_up == in_sky_is_up_sector:
            return roll
```

```
        if roll < FULL_ROTATION / 2:
            roll += FULL_ROTATION / 2
        else:
            roll -= FULL_ROTATION / 2

        return roll
```

4 Exemple

4.1 Orizont detectabil

In primul caz, imaginea contine o separatie clara intre cer si teren, iar linia orizontului este detectata corect prin metoda descrisa.

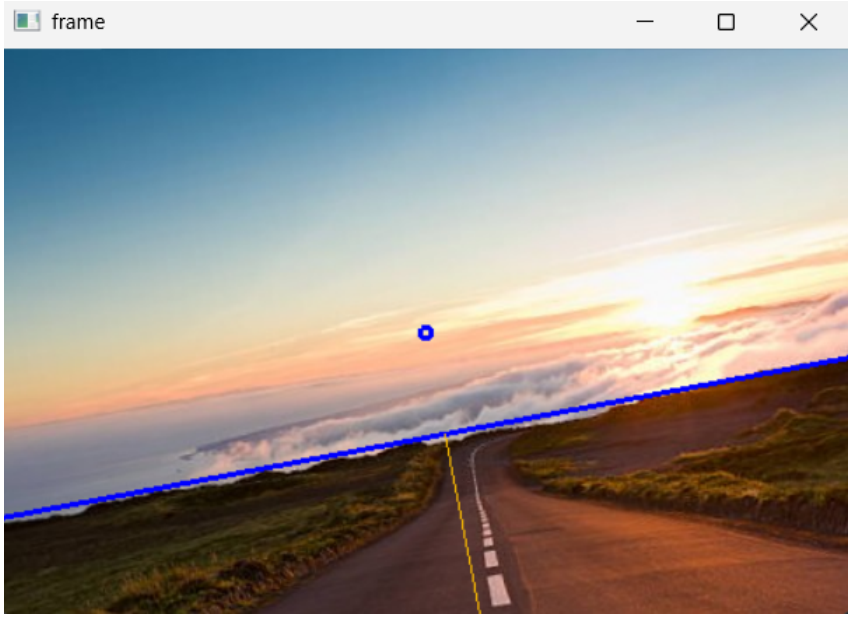

Figure 1: Exemplu cu orizont detectat. Fit-ul separa corect cerul de teren.

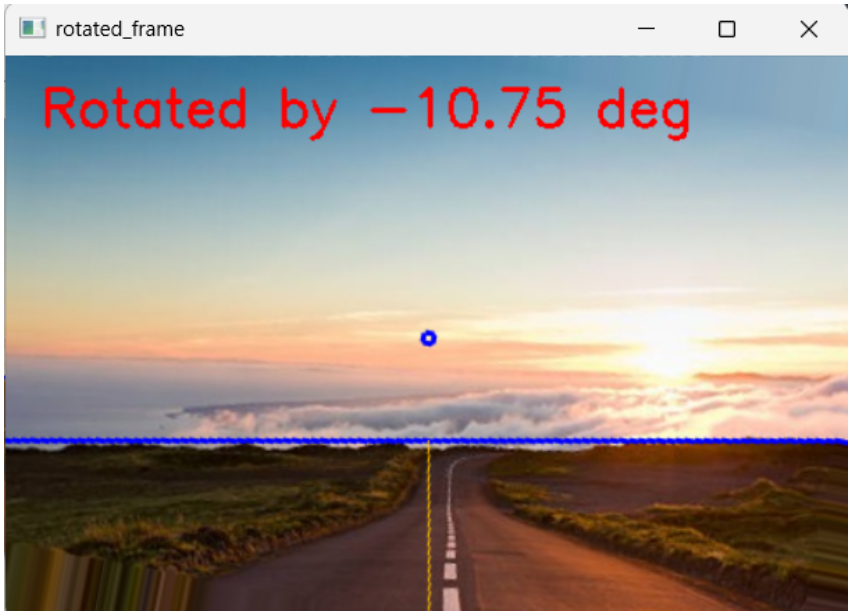

Figure 2: Imaginea corectata conform parametrilor de roll si pitch estimati.

4.2 Orizont nedetectabil

In al doilea caz, conditiile nu permit o separare distincta (de ex: cer acoperit complet sau zgomot excesiv), astfel incat algoritmul nu reuseste sa stabileasca o linie valida.

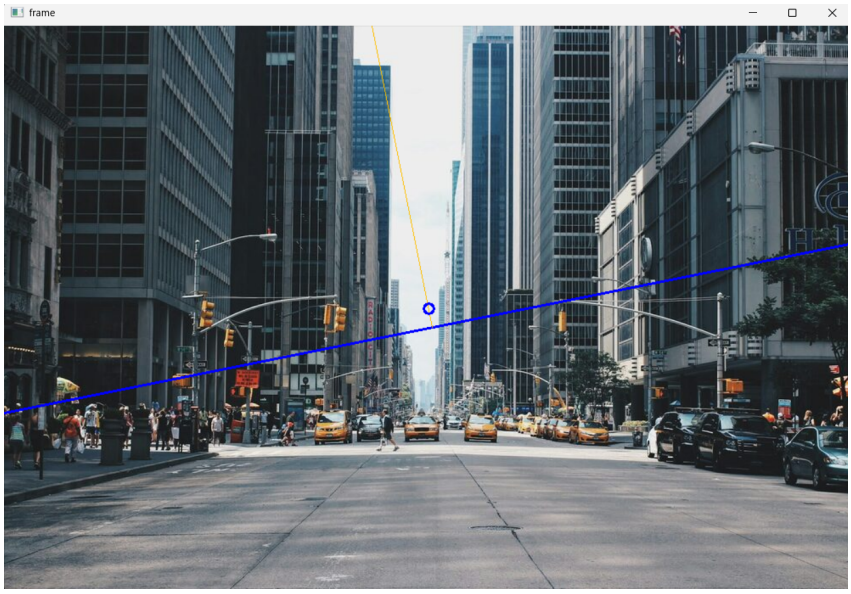

Figure 3: Exemplu fara orizont. Algoritmul nu gaseste raspuns. Traseaza o linie fictiva drept incercare in zadar.